

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Cloud Image Registry & Distribution

Data 2026/5/28 Class AI 1

Name Weijie Xu ID 202383910029

一、Experimental purpose

(1) Practice Docker Image Version Control

The first purpose of this experiment is to understand how Docker images can be managed through version control in a cloud computing environment. By building different image versions, such as v1.0 and v2.0, this experiment helps us learn how Docker tags are used to distinguish different iterations of the same application. This is important because cloud applications are usually updated continuously, and each version must be clearly identified, stored, and deployed. Through this process, we can understand that an image tag is not only a name, but also a practical mechanism for managing application lifecycle, rollback, testing, and release control.

(2) Understand Cloud Image Registry and Distribution

The second purpose is to learn how a container image can be uploaded to and distributed through a cloud image registry. In this experiment, GitHub Container Registry is used as the remote storage platform for Docker images. By tagging the local image in the required registry format and pushing it to GHCR, we can understand how a local application package becomes a cloud-distributable resource. This process shows the basic workflow of cloud-native software delivery, where applications are packaged once and then distributed to different environments through a centralized registry.

(3) Master Secure Authentication with Personal Access Tokens

The third purpose is to practice secure authentication when interacting with cloud platforms. Instead of using a plaintext GitHub password in the terminal, this experiment requires the use of a Personal Access Token. This helps us understand the importance of secret management in cloud computing. A token can be given limited permissions, such as package writing and reading, and can be revoked when necessary. Therefore, using PAT improves security and reduces the risk caused by exposing the

main account password in command-line operations or cloud development environments.

(4) Observe Docker Layer Caching and Incremental Update Mechanism

The fourth purpose is to observe how Docker uses layered storage to improve image transmission efficiency. After modifying the web application from version v1.0 to version v2.0, we rebuild and push the new image to the remote registry. During this process, some layers may show “Layer already exists”, which means that Docker does not upload unchanged layers repeatedly. Only the modified layer needs to be uploaded. This experiment helps us understand why container technology is efficient for cloud deployment, especially when applications are frequently updated but most dependencies and base layers remain unchanged.

(5) Validate Decoupled Deployment and Disaster Recovery Ability

The fifth purpose is to verify that a containerized application can be deployed independently of the original local development environment. By deleting local images and containers and then pulling the image from the cloud registry, we simulate a clean-room deployment or disaster recovery scenario. This proves that the application is no longer tied to local source code files or the original machine. As long as the image is stored in the cloud registry, it can be pulled and started on a new server. This is a key idea in cloud computing because it supports portability, reliability, and fast recovery.

(6) Compare Image Optimization Strategies for Cloud Deployment

The sixth purpose is to understand how image size affects cloud deployment efficiency. The experiment compares standard Nginx-based images with Alpine-based images to observe the difference in storage size. Since smaller images can reduce storage costs, network transfer time, and cold start delay, image optimization is an important practice in production cloud environments. Through this comparison, we can understand why lightweight base images such as Alpine are widely used in microservice deployment. This part also helps us build awareness of performance optimization, resource efficiency, and practical engineering choices in cloud computing.

二、Experiment Contents

- In this experiment, we first completed secure authentication between the local cloud development environment and GitHub Container Registry. We entered the GitHub account settings, created a Personal Access Token with the required package-related permissions, and used this token to log in to ghcr.io through the

terminal. This step allowed Docker to communicate with the remote container registry securely. During this process, we learned that cloud platforms usually do not recommend using account passwords directly in terminals or automated scripts. Instead, tokens with limited permissions are used to improve security and make cloud operations more controllable.

- After finishing authentication, we built a Docker image for the web application based on the previous laboratory project. The local image was named with a clear version label, such as `cloud-lab-image`. This step helped us understand how source code and runtime configuration can be packaged into a container image. The image contains all necessary files and environment settings required to run the application, so the application can be started consistently in different environments. Through this operation, we practiced the basic process of transforming a local web project into a portable cloud application package.
- We then tagged the local Docker image according to the naming rules of GitHub Container Registry. The original local image name was converted into a registry-compatible format, such as `ghcr.io/username/cloud-app.0`. This step was necessary because a cloud registry needs a complete image address to identify the owner, repository name, application name, and version number. By applying this tag, the image became ready to be pushed to the cloud. This part of the experiment strengthened our understanding of image naming conventions and showed that correct tagging is an important bridge between local development and remote distribution.
- After tagging the image, we pushed the `v1.0` image to GitHub Container Registry. This operation uploaded the packaged application from the local environment to GitHub's cloud infrastructure. After the push command finished successfully, we checked the GitHub Packages page to verify whether the `cloud-app` package appeared under the GitHub profile. This step simulated the "ship" phase in cloud software delivery, where the application is no longer stored only on the developer's machine but becomes a remotely accessible image resource. It also helped us understand the role of image registries in cloud-native deployment pipelines.
- Next, we performed a version iteration from `v1.0` to `v2.0`. We modified the web page content, such as changing the displayed text or visual style, then rebuilt the Docker image and assigned a new tag to it. After pushing the `v2.0` image to the registry, we observed the terminal output and noticed that some layers already existed. This phenomenon showed Docker's layered storage and cache mechanism. Since only the modified application layer needed to be uploaded again, the second push was

faster than the first push. This step helped us understand why Docker is efficient for frequent cloud application updates.

- We also carried out a disaster recovery and clean-room deployment test. To simulate a fresh server environment, we removed local images and containers, then pulled and ran the application image directly from the cloud registry. This operation proved that the application could still be deployed successfully even when the original local image and source code were no longer available. We also tested the situation where another container attempted to use the same host port, which caused a port allocation conflict. Through this part, we learned that cloud deployment requires not only image availability but also correct port management and runtime configuration.
- Finally, we compared different image optimization strategies and saved the experimental changes with Git. We created and built both the standard Nginx-based image and the Alpine-based image, then compared their sizes through Docker commands. This comparison helped us understand how the choice of base image affects storage cost, network transmission time, and deployment speed. Since Alpine images are usually smaller, they are widely used in production microservice environments. At the end of the experiment, we used `git add`, `git commit`, and `git push` to save the modified files and experimental results to GitHub, ensuring that the laboratory work could be permanently recorded and tracked.

三、 Detailed steps of the experiment

Step 1.1: Open GitHub Developer Settings

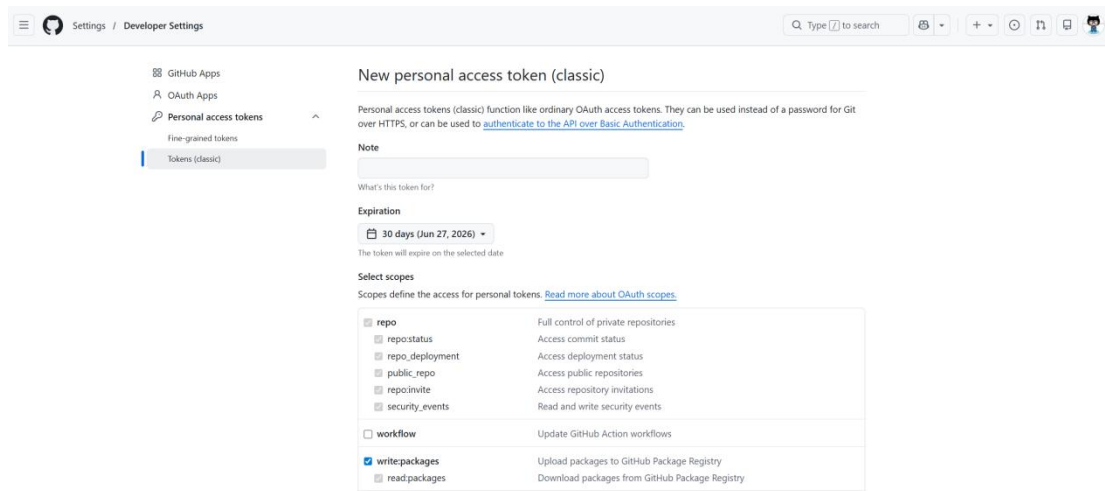
This screenshot records the beginning of secure authentication setup. I opened GitHub account settings and entered Developer settings, where GitHub allows users to manage automation credentials such as Personal Access Tokens. This step is necessary because Docker needs permission to communicate with GitHub Container Registry from the terminal. In a cloud platform, account passwords should not be used directly for automated tools. Instead, a limited token can represent the user for a specific task. The cloud concept shown here is secure identity management, because access to cloud resources must be controlled before images can be pushed to a registry.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker --version
Docker version 29.3.0-1, build 5927d80c76b3ce5cf782be818922966e8a0d87a3
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ ls
Dockerfile  index.html
```

Step 1.2: Generate a Personal Access Token

After entering the Personal access tokens area, I selected Tokens classic and started generating a new token. This screenshot shows the process of creating a credential

that can be used by command-line tools and automation scripts. A Personal Access Token is different from a normal password because it can be created for a specific purpose and revoked later without changing the whole account login. This is important in cloud computing because many operations are performed by terminals, pipelines, or services. The token provides controlled machine access to GitHub Container Registry while keeping the main GitHub account safer.



Step 1.3: Select Required Token Permissions

In this step, I selected the required token permission, especially write:packages. This permission allows Docker to push container images to GitHub Container Registry, and it also includes the related package reading permission. Choosing the correct scope is important because a token should have enough permission to complete the task but should not be granted unnecessary access. This follows the principle of least privilege in cloud security. The screenshot demonstrates how cloud platforms divide access into different scopes, so users can control exactly what an automation credential is allowed to do.

```
• @qzxwj → /workspaces/cloud-computing-lab-github (main) $ export CR_PAT=ghp_EOCpX30Qrh15YlQH2QU1EJFcZEBR0o2RsiYS
• @qzxwj → /workspaces/cloud-computing-lab-github (main) $ echo $CR_PAT | docker login ghcr.io -u qzxwj --password-stdin

WARNING! Your credentials are stored unencrypted in '/home/codespace/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
```

Step 1.4: Save the Generated Token

After generating the token, I copied it immediately and saved it temporarily for terminal login. GitHub only displays the token once, so it must be copied at creation time. This step is important because losing the token means it cannot be viewed again and must be regenerated. At the same time, the token should be protected carefully because anyone who obtains it may be able to access package resources within its

permission scope. From a cloud security perspective, this shows how secret management works: credentials are powerful, time-sensitive values that should be copied only when needed and stored securely.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-lab-image:v1 .
[+] Building 5.6s (8/8) FINISHED
=> [internal] load build definition from Dockerfile                                docker:default 0.0s
=> => transferring dockerfile: 314B                                              0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine                  2.3s
=> [auth] library/nginx:pull token for registry-1.docker.io                     0.0s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                  0.0s
=> [internal] load build context                                                0.0s
=> => transferring context: 1.17kB                                              0.0s
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a 1.9s
=> => resolve docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a 0.0s
=> => sha256:fbaed3f7fcb6a0d591ac8601934f1f05252cee42741fde9e950dce55580af18 20.28MB / 20.28MB 0.7s
=> => sha256:a5008f4a4b257356728e2feaf2b5b9e2054c0c577d324e76abbf14470b5f1cfd 1.40kB / 1.40kB 0.7s
=> => sha256:e654dbbbb9e15a1fa035d24728aeeb60bc655f4ebd4d6215a496b77a7d104697 1.21kB / 1.21kB 0.7s
=> => sha256:94d083cf706ab544ec7e9bcaba5c164db87b3d3a56176bf2fca440d535c16b0d 405B / 405B 0.7s
=> => sha256:de1b677d8c003ce9e558f3a0cd4dec3c035f424ecddd68063043a593ad572257 957B / 957B 0.3s
=> => sha256:43f834d60d8af3f133c7e76a202d28cd62cc026a561edca72ee752ef01bfaacd 628B / 628B 0.3s
```

Step 1.5: Login to GitHub Container Registry

In the Codespaces terminal, I exported the token as `CR_PAT` and used `docker login ghcr.io` with `--password-stdin` to authenticate with GitHub Container Registry. Passing the token through standard input is safer than typing it directly into the command line because it reduces the chance of exposing the secret in shell history. The expected observation is Login Succeeded. A warning about credentials being stored unencrypted may appear in a cloud IDE environment, but the successful login confirms that Docker can now push and pull images from GHCR. This step connects local container tooling with cloud registry authentication.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker images
INFO → UI In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
cloud-lab-image:v1 e399bea8f19b 92.7MB 26MB
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker tag cloud-lab-image:v1 ghcr.io/qzxwj/cloud-app:v1.0
```

Step 2.1: Build the Local Docker Image

Before pushing an image to the cloud registry, I built the local Docker image based on the previous lab project. The command `docker build -t cloud-lab-image:v1 .` reads the Dockerfile in the current directory and packages the web application together with its runtime environment. The local tag `cloud-lab-image:v1` helps identify this first version clearly. This step prepares the application as a container image, which is a portable deployment artifact. In cloud computing, building the image locally or in a cloud workspace is the packaging stage before distribution to a remote image registry.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker images
INFO → UI In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
cloud-lab-image:v1 e399bea8f19b 92.7MB 26MB
ghcr.io/qzxwj/cloud-app:v1.0 e399bea8f19b 92.7MB 26MB
```

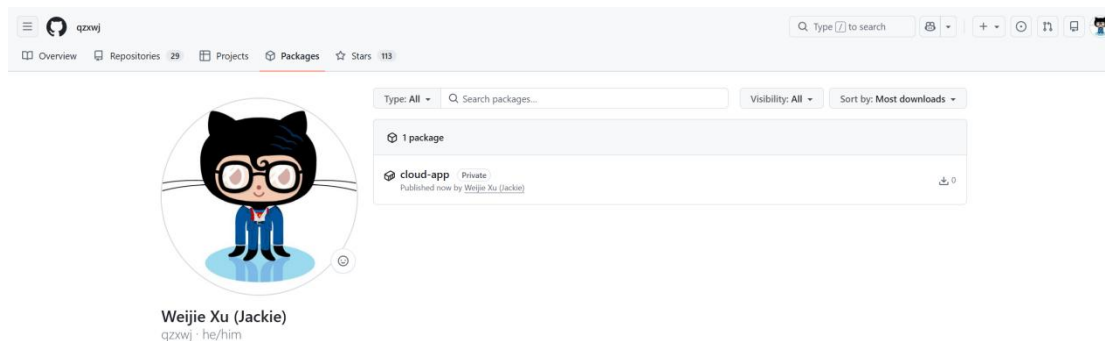
Step 2.2: Tag the Image for GHCR

After the image was built locally, I applied a registry-compatible tag with the format `ghcr.io/username/cloud-app:v1.0`. This step converts the local image name into a cloud-ready image address. The registry domain `ghcr.io` tells Docker where the image will be pushed, the username identifies the owner namespace, `cloud-app` is the package name, and `v1.0` is the version tag. GitHub usernames in this path should be lowercase to avoid naming problems. The cloud concept here is image addressing. A registry must use a strict naming convention so images can be stored, discovered, versioned, and pulled correctly.

```
• @qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker push ghcr.io/qzxwj/cloud-app:v1.0
The push refers to repository [ghcr.io/qzxwj/cloud-app]
a5008f4a4b25: Pushed
abaae85d1626: Pushed
fbaed3f7fcbe: Pushed
6a0ac1617861: Pushed
43f834d60d8a: Pushed
4e95bfec6812: Pushed
770b71d246d2: Pushed
de1b677d8c00: Pushed
94d083cf706a: Pushed
e654dbbbb9e1: Pushed
v1.0: digest: sha256:e399bea8f19bcebf4ea8d02cfdb8d2ed40e615a20c390c108efdefd4a1532f3d size: 856
```

Step 3.1: Push the v1.0 Image

In this step, I used `docker push ghcr.io/username/cloud-app:v1.0` to upload the tagged image to GitHub Container Registry. This is the ship phase of the container workflow. Docker checks the image layers, uploads missing layers, and registers the version tag in the remote registry. Once the push finishes successfully, the image is no longer limited to the current Codespaces environment. It becomes a cloud-hosted package that can be downloaded and deployed by other machines with permission. This demonstrates image distribution through global cloud infrastructure, where a packaged application can be stored centrally and reused later.



Step 3.2: Verify the Package on GitHub

After pushing the image, I opened my GitHub profile and checked the Packages area to verify that cloud-app appeared. This screenshot confirms that the image upload was successful from the web interface, not only from terminal output. Verification is important because cloud operations should be checked from both the command-line result and the platform dashboard. The package page shows that GHCR has stored the container image as a managed registry resource. This step demonstrates how cloud platforms provide visibility into deployed artifacts, including package names, versions, ownership, and access settings.

```
<body>
  <h1>Welcome to Version 2.0</h1>

  <p>This site was deployed using <b>GitHub Actions</b>.</p>

  <div class="status">• Site Status: Live</div>

  <hr>

  <p>Student Name: Xu Weijie AI Class 1</p>
  <p>Current Cloud Provider: GitHub (Microsoft Azure Infrastructure)</p>
</body>
```

Step 4.1: Modify the Web Page for v2.0

To create a new version, I opened index.html and changed a visible element, such as the background color or displayed text to Welcome to Version 2.0. This modification represents a normal application iteration. The purpose is to create a meaningful difference between v1.0 and v2.0 so that the later rebuild and push process can be observed clearly. In cloud application development, services are updated continuously, but each version should still be traceable. This step prepares the source code change that will become a new container image version, showing how application content changes are converted into versioned deployment artifacts.

```
● @qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-lab-image:v2 .
[+] Building 1.5s (8/8) FINISHED
=> [internal] load build definition from Dockerfile                                docker:default
=> => transferring dockerfile: 314B                                              0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine                  1.1s
=> [auth] library/nginx:pull token for registry-1.docker.io                     0.0s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                  0.0s
=> [internal] load build context                                                0.0s
=> => transferring context: 1.16kB                                              0.0s
=> CACHED [1/2] FROM docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a 0.0s
=> => resolve docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a 0.0s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html                     0.0s
=> exporting to image                                                            0.2s
=> => exporting layers                                                            0.1s
=> => exporting manifest sha256:cca8227bfff51b71737451cee989d4f4492ae16dd057a3c44887b453534505dd7 0.0s
=> => exporting config sha256:3f3703fc419abee14e3a65e14604851c56b47e5cfe2ab26387fa6ea585707d8a 0.0s
=> => exporting attestation manifest sha256:5bc7897ac49766acbae62a66080800b77e4a8c7619169b670c2d5cfba4ad3036 0.0s
```

Step 4.2: Rebuild the v2.0 Image

After modifying the web page, I rebuilt the Docker image with a new local tag, such as cloud-lab-image:v2. The build process reads the same Dockerfile again but

includes the updated index.html content. Docker uses its layer cache to avoid repeating unchanged work when possible. For example, if the base Nginx image and earlier instructions are unchanged, Docker can reuse those layers and only recreate the layer affected by the modified HTML file. This step demonstrates incremental image building, which is important for efficient cloud development because frequent application updates should not require rebuilding everything from zero.

```
● @qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker push ghcr.io/qzxwj/cloud-app:v2.0
The push refers to repository [ghcr.io/qzxwj/cloud-app]
6a0ac1617861: Layer already exists
abaae85d1626: Layer already exists
de1b677d8c00: Layer already exists
94d083cf706a: Layer already exists
cdd3f09b8493: Pushed
957c43b93f73: Pushed
43f834d60d8a: Layer already exists
e654dbbbb9e1: Layer already exists
a5008f4a4b25: Layer already exists
fbaed3f7fcbe: Layer already exists
v2.0: digest: sha256:5a7e518b800803ea19a3f762cb5330322635d3f02479c0b61a049a90acd3783a size: 856
```

Step 4.3: Retag the v2.0 Image

After building the updated local image, I tagged it as ghcr.io/username/cloud-app:v2.0. This creates a second cloud registry address for the new application version. The tag v2.0 distinguishes it from v1.0, allowing both versions to exist in the registry at the same time. Version tags are important because they allow controlled deployment and rollback. In a cloud environment, users may need to run an older stable version or test a newer release independently. This step shows how Docker image tags support version management, release tracking, and repeatable deployment.

```
● @qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker system prune -a
\WARNING! This will remove:
- all stopped containers
- all networks not used by at least one container
- all images without at least one container associated to them
- all build cache

Are you sure you want to continue? [y/N] y
```

Step 4.4: Push v2.0 and Observe Layer Reuse

I pushed the v2.0 image to GitHub Container Registry and observed the terminal output. Some lines reported Layer already exists, which means Docker did not upload every layer again. Since most of the image content, such as the Nginx base layer, stayed the same, the registry only needed to receive the changed layer related to the updated web content. This is Docker's layer reuse mechanism. The cloud concept is efficient incremental distribution. By uploading only changed layers, Docker saves time, bandwidth, and storage, which is especially valuable when cloud applications are updated frequently.

```
@qzxcwj → /workspaces/cloud-computing-lab-github (main) $ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
cloud-lab-image:v1	e399bea8f19b	92.7MB	26MB	
cloud-lab-image:v2	5a7e518b8008	92.7MB	26MB	
ghcr.io/qzxcwj/cloud-app:v1.0	e399bea8f19b	92.7MB	26MB	
ghcr.io/qzxcwj/cloud-app:v2.0	5a7e518b8008	92.7MB	26MB	

```
@qzxcwj → /workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8888:80 ghcr.io/qzxcwj/cloud-app:v1.0
d04ebd855ec2d09baf304aa36d530ab41b013a39dc5117827b1b97f39b3cd034
```

Step 5.1: Remove Local Images and Containers

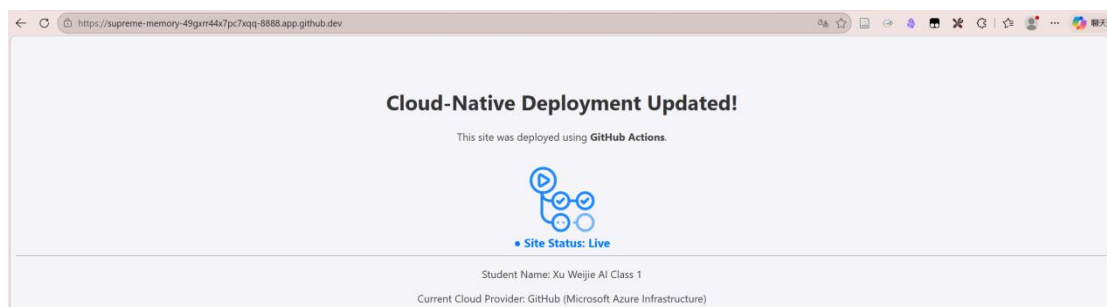
To simulate a new server or disaster recovery environment, I removed local containers and images with `docker system prune -a`. This command clears Docker resources from the current environment, so the application can no longer rely on locally built images. This step must be used carefully because it deletes local Docker data. In the experiment, the cleanup proves whether the application has really been stored in the cloud registry. The cloud concept is decoupling from the local machine. If the image is safely stored in GHCR, the service can be restored later even after local image data is removed.

```
@qzxcwj → /workspaces/cloud-computing-lab-github (main) $ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
d04ebd855ec2	ghcr.io/qzxcwj/cloud-app:v1.0	"/docker-entrypoint..."	23 seconds ago	Up 22 seconds	0.0.0.0:8888->80/tcp, [::]:8888->80/tcp

Step 5.2: Deploy v1.0 from the Cloud Registry

After removing local Docker data, I ran `docker run -d -p 8888:80 ghcr.io/username/cloud-app:v1.0` to deploy the application directly from GitHub Container Registry. Since the local image was no longer available, Docker pulled it from the remote registry before starting the container. This proves that the packaged application is independent of the original source files and the previous local build cache. In cloud computing, this is similar to disaster recovery or clean-room deployment. A new server can pull a trusted image from a registry and start the service without rebuilding from source code.



Step 5.3: Verify the Service on Port 8888

After running the container from the remote image, I opened the Codespaces Ports tab

and accessed the service through port 8888. The successful page display proves that the image pulled from GHCR can run normally and provide the web service. This verification is important because a successful docker run command alone does not always guarantee that the application is reachable from the browser. The screenshot connects registry storage, remote image pulling, container startup, port mapping, and browser access into one complete deployment result. It shows that cloud-hosted images can be used to restore a working service quickly.

```
@qzxxwj → /workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8888:80 ghcr.io/honglinyuan/cloud-app:v2.0
Unable to find image 'ghcr.io/honglinyuan/cloud-app:v2.0' locally
docker: Error response from daemon: unknown: failed to resolve reference "ghcr.io/honglinyuan/cloud-app:v2.0": unexpected status from HEAD
request to https://ghcr.io/v2/honglinyuan/cloud-app/manifests/v2.0: 403 Forbidden

Run 'docker run --help' for more information
```

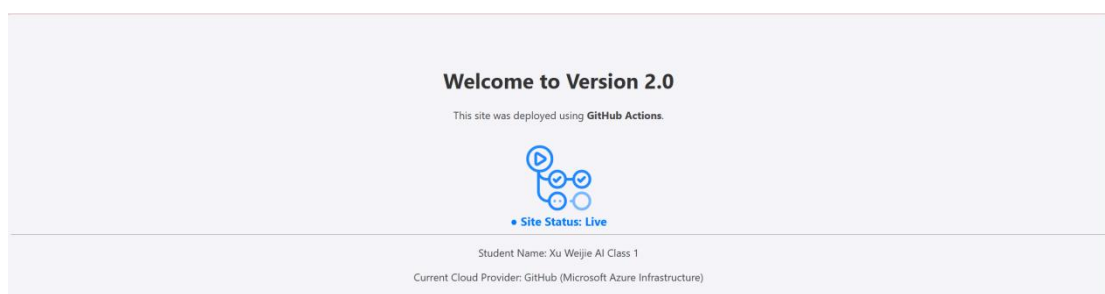
Step 5.4: Reproduce the Port Conflict

Next, I attempted to run another container on the same host port 8888. Docker returned an error similar to Bind for 0.0.0.0:8888 failed: port is already allocated. This happens because the host port is already occupied by the first running container. Even if the second image is valid, the host cannot bind two services to the same port at the same time. This step intentionally reproduces a common cloud deployment problem. It shows that container images solve packaging and portability, but engineers still need to manage runtime resources such as ports, names, and network mappings.

```
@qzxxwj → /workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8889:80 ghcr.io/qzxxwj/cloud-app:v2.0
164adca0bd2d7807440bab7a959d8d319a980cfc7d1c0ea51bd9a284a32e91a4
```

Step 5.5: Solve the Port Conflict

To solve the port conflict, I used practical container management methods. One method is to stop and remove the existing container that uses port 8888, then start the new container on the same port. Another method is to keep the first container running and map the second container to a different host port, such as 8889:80. In larger cloud systems, a load balancer or reverse proxy can also route traffic to multiple container instances without exposing each container on the same host port. This step explains the operational concept behind the error: scaling is possible, but network entry points must be planned and managed correctly.



Step 6.1: Create Dockerfile.alpine

For image optimization, I created a file named Dockerfile.alpine and used nginx:alpine as the base image. This file copies the web project into the Nginx HTML directory just like the standard Dockerfile, but it relies on the Alpine-based Nginx image. Alpine Linux is designed to be small and lightweight, so images based on it are usually much smaller than standard Linux-based images. This step prepares the comparison between different base images. In cloud computing, base image selection matters because image size affects storage cost, network transfer time, and deployment startup speed.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-app:standard -f Dockerfile .
[+] Building 1.6s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 314B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [auth] library/nginx:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 32B
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a
docker:default
0.0s
0.0s
1.4s
0.0s
0.0s
0.0s
0.0s
0.0s
```

Step 6.2: Build Standard and Alpine Images

I built two image versions for comparison: cloud-app:standard with the normal Dockerfile and cloud-app:alpine with Dockerfile.alpine. Using the -f option allows Docker to select a specific Dockerfile for the build. Both images can provide the same simple web service, but they may have different sizes because their base operating system layers are different. This step demonstrates that two containers can behave similarly at the application level while having different resource characteristics. In production cloud environments, engineers often compare image bases to balance compatibility, security, size, and deployment performance.

```
• @qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-app:alpine -f Dockerfile.alpine .
[+] Building 0.9s (7/7) FINISHED
=> [internal] load build definition from Dockerfile.alpine
=> => transferring dockerfile: 90B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 53.38kB
docker:default
0.0s
0.0s
0.3s
0.0s
0.0s
0.1s
0.0s
```

Step 6.3: Compare Image Sizes

After building and pulling the relevant Nginx images, I used docker images and compared the sizes of nginx:latest, nginx:alpine, cloud-app:standard, and cloud-app:alpine. The report should record the exact sizes shown in the terminal. The expected result is that Alpine-based images are much smaller because Alpine contains fewer system components and a minimal user-space environment. This is why Alpine

is widely used for production microservices when the application does not require a full operating system image. Smaller images reduce registry storage usage, speed up pulling, and improve cold start performance in cloud deployment.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker pull nginx:latest
latest: Pulling from library/nginx
13fd728be9eb: Pull complete
5431d0092ffd: Pull complete
5b4d6ff92fc4: Pull complete
830625e1ac85: Pull complete
7f8b1a2b17d8: Pull complete
b4a248c845e5: Pull complete
45381ecb0e2f: Pull complete
5e6b66b5e5f1: Download complete
cc5f57206478: Download complete
Digest: sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest

@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker pull nginx:alpine
alpine: Pulling from library/nginx
Digest: sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a
Status: Downloaded newer image for nginx:alpine
docker.io/library/nginx:alpine

@qzxwj → /workspaces/cloud-computing-lab-github (main) $ docker images | grep nginx
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
nginx:alpine          8b1e78743a03          93.6MB          27MB
nginx:latest          5aca99593157          241MB          66MB
```

Step 7: Save Changes with Git

Finally, I used Git commands to save the lab changes back to the remote repository. The git add command stages the modified files, git commit records them as a versioned snapshot with a message such as Finished Lab 03, and git push uploads the commit to GitHub. This step ensures that the Dockerfiles, updated web page, and lab-related changes are preserved in the cloud repository. It also completes the full workflow of the experiment: authenticate securely, package the application as images, push versions to a registry, test recovery and optimization, and then store the source changes permanently through version control.

```
@qzxwj → /workspaces/cloud-computing-lab-github (main) $ git push origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 579 bytes | 579.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/qzxwj/cloud-computing-lab-github
e9d57b8..1bea876 main -> main
```

四、Reflection

1. Problems Encountered and Proposed Solutions

(1) During the authentication stage, one possible problem was that the Docker login command failed because the Personal Access Token was copied incorrectly or did not include the required permission. Since GitHub only shows the token once after generation, it is easy to lose or paste an incomplete token. To solve this problem, I needed to regenerate a new token in GitHub Developer settings and make sure that the write permission was selected. After that, I exported the token again as an

environment variable and used the password-stdin method to log in to ghcr.io. This solution avoided typing the token directly in the command and made the authentication process safer.

(2) Another problem was related to image tagging and naming rules. GitHub Container Registry requires a specific image name format, and the GitHub username in the image path should be lowercase. If the image was tagged with an incorrect username, repository name, or version label, the push command could fail or the package might not appear correctly in GitHub Packages. To solve this problem, I checked the image name carefully before pushing it and used the correct format, such as ghcr.io/username/cloud-app.0. This helped me understand that cloud registries rely on strict naming conventions to identify image ownership, application name, and version information.

(3) A further problem occurred during the deployment stage when running another container on the same host port. For example, if port 8888 had already been occupied by a running container, Docker would report that the port was already allocated. This means that two containers cannot bind to the same host port at the same time. To solve this problem, I could stop and remove the previous container before starting the new one, or map the second container to another available port, such as 8889:80. This problem helped me understand that successful cloud deployment depends not only on pulling the correct image but also on managing ports, containers, and runtime resources properly.

2. Efficiency: Why did Step 4.3 upload much faster than Step 3.1? Relate this to the concept of Layer Caching.

(1) Step 4.3 uploaded much faster than Step 3.1 because Docker images are built with a layered structure. When the v1.0 image was pushed for the first time, Docker needed to upload all image layers to the remote registry, including the base image layer, dependency layer, configuration layer, and application files. However, when the application was modified and rebuilt as v2.0, most of these layers remained unchanged. Docker compared the local image layers with the layers already stored in GitHub Container Registry and found that many layers already existed. Therefore, Docker skipped these unchanged layers and only uploaded the new or modified layer.

(2) Layer caching improves efficiency because Docker identifies each layer through its content-based identifier. If the content of a layer does not change, its identifier also remains the same. In this experiment, the main modification was usually a small

change in index.html, such as changing the page text or background color. This change only affected the application content layer, while the Nginx base image and other unchanged layers could be reused directly. As a result, the second push required much less data transmission. This is why the terminal output showed messages such as “Layer already exists”, which means Docker was reusing previously uploaded layers instead of transferring the whole image again.

(3) This mechanism is very important in cloud computing because cloud applications are updated frequently. If every update required uploading the complete image again, deployment would waste time, bandwidth, and storage resources. With layer caching, only the changed part of the application needs to be uploaded, which makes continuous integration and continuous deployment more efficient. It also reduces the cost of image distribution in large-scale cloud environments. Through this experiment, I understood that Docker’s layered storage is not only a technical detail, but also a key reason why container-based deployment is fast, scalable, and suitable for modern cloud-native development.

3. Security: Why is using a Personal Access Token more secure than using your actual GitHub password in a terminal?

(1) Using a Personal Access Token is more secure because it can be granted limited permissions, while a GitHub account password usually provides broad access to the entire account. In this experiment, the token only needed package-related permissions, such as writing and reading packages. This means that even if the token was accidentally exposed, the potential damage would be limited to the allowed scope. By contrast, exposing the real GitHub password could put the whole account at risk, including repositories, settings, personal information, and other connected services. Therefore, PAT follows the principle of least privilege in cloud security.

(2) A Personal Access Token is also easier to manage and revoke. If a token is no longer needed or suspected to be leaked, it can be deleted directly from GitHub settings without changing the account password. This is especially useful in cloud development environments, automation scripts, and CI/CD pipelines, where credentials may be used by tools rather than manually entered by users. In this experiment, using a token made the Docker login process safer and more suitable for automated cloud operations. It allowed authentication without exposing the main password in the terminal command history or configuration files.

(3) Another security advantage is that PAT-based authentication separates human login from machine access. A GitHub password is mainly designed for direct user login, while a token is designed for controlled programmatic access. Cloud platforms, container registries, and deployment tools often need non-interactive authentication, so tokens are more appropriate for these scenarios. In addition, tokens can be created for specific tasks and removed after the task is completed. This reduces long-term security risks and makes access control more flexible. Through this part of the experiment, I learned that secure credential management is a basic requirement in cloud computing, especially when working with remote registries and automated deployment workflows.

附录 程序命令代码

附录 1: docker_command

```
# =====  
# Step 0: Check Environment  
# =====  
  
# Check Docker version  
docker --version  
  
# Check files in current directory  
ls  
  
# If needed, enter your project folder  
cd your-project-folder  
  
# =====  
# Step 1: Secure Authentication with GHCR  
# =====  
  
# Linux / macOS / Git Bash / Codespaces  
export CR_PAT=YOUR_COPIED_TOKEN  
  
# Login to GitHub Container Registry  
echo $CR_PAT | docker login ghcr.io -u YOUR_USERNAME --password-stdin
```

```
# =====
# Windows PowerShell Version
# Only use this part if you are using PowerShell
$env:CR_PAT="YOUR_COPIED_TOKEN"
echo $env:CR_PAT | docker login ghcr.io -u YOUR_USERNAME --password-stdin

# =====
# Step 2: Build and Tag v1.0 Image
# =====

# Build local Docker image
docker build -t cloud-lab-image:v1 .

# Check local images
docker images

# Tag the local image into GHCR format
# Note: YOUR_USERNAME should be lowercase
docker tag cloud-lab-image:v1 ghcr.io/YOUR_USERNAME/cloud-app:v1.0

# Check whether the new tag exists
docker images

# =====
# Step 3: Push v1.0 Image to GHCR
# =====

# Push v1.0 image to GitHub Container Registry
docker push ghcr.io/YOUR_USERNAME/cloud-app:v1.0

# =====
# Step 4: Modify Code, Rebuild, Retag, and Push v2.0
# =====
```

```
# After manually modifying index.html, rebuild the image
docker build -t cloud-lab-image:v2 .

# Tag v2 image into GHCR format
docker tag cloud-lab-image:v2 ghcr.io/YOUR_USERNAME/cloud-app:v2.0

# Push v2.0 image to GHCR
docker push ghcr.io/YOUR_USERNAME/cloud-app:v2.0

# =====
# Step 5: Disaster Recovery Test
# =====

# Warning: This removes unused local containers, networks, images, and cache
docker system prune -a

# Check local images after cleanup
docker images

# Pull and run v1.0 image directly from GHCR
docker run -d -p 8888:80 ghcr.io/YOUR_USERNAME/cloud-app:v1.0

# Check running containers
docker ps

# Try to run another container on the same port
# This may cause: port is already allocated
docker run -d -p 8888:80 ghcr.io/YOUR_USERNAME/cloud-app:v2.0

# Solution 1: Stop and remove the old container
docker ps
docker stop CONTAINER_ID
docker rm CONTAINER_ID

# Then run v2.0 again on port 8888
```

```
docker run -d -p 8888:80 ghcr.io/YOUR_USERNAME/cloud-app:v2.0
```

```
# Solution 2: Keep the old container and use another host port
```

```
docker run -d -p 8889:80 ghcr.io/YOUR_USERNAME/cloud-app:v2.0
```

```
# =====
```

```
# Step 6: Image Optimization - Standard vs Alpine
```

```
# =====
```

```
# Build the standard version based on Dockerfile
```

```
docker build -t cloud-app:standard -f Dockerfile .
```

```
# Build the Alpine version based on Dockerfile.alpine
```

```
docker build -t cloud-app:alpine -f Dockerfile.alpine .
```

```
# Pull official Nginx images for comparison
```

```
docker pull nginx:latest
```

```
docker pull nginx:alpine
```

```
# Compare Nginx image sizes
```

```
docker images | grep nginx
```

```
# Compare your own cloud-app image sizes
```

```
docker images | grep cloud-app
```

```
# =====
```

```
# Windows PowerShell Alternative for Image Checking
```

```
# Use this if grep does not work
```

```
# =====
```

```
docker images
```

```
# =====
```

```
# Step 7: Save Changes with Git
```

```
# =====
```

```
# Check current Git status
```

```
git status
# Add all changes
git add .

# Commit changes
git commit -m "Finished Lab 03"

# Push changes to GitHub
git push

# =====
# Optional: If the project has not been initialized with Git
# =====

git init
git remote add origin
https://github.com/YOUR_USERNAME/YOUR_REPOSITORY.git
git branch -M main
git add .
git commit -m "Finished Lab 03"
git push -u origin main
```